

Orbital Debris Risk Analysis & Conjunction Monitoring

Project sample prepared for a data analyst application

Summary of a Streamlit-based orbital dashboard that cleans, structures, and analyzes live TLE data to identify potential conjunction risks.

Project type	Analytics case study / dashboard
Primary stack	Python, pandas, Skyfield, Folium, Streamlit
Data source	Live active-satellite TLE feed stored in active.txt (~14,683 records)
Core output	Interactive map and risk alerts for close-proximity satellite pairs

Executive Summary

This project analyzes orbital conjunction risk using a real-time satellite dataset and presents the results in an interactive dashboard. The raw TLE catalogue is parsed into structured satellite records, transformed into analysis-ready coordinates, and screened for near-miss events using a three-dimensional distance calculation. The end result is a concise visual tool that helps users understand where congestion exists and which objects deserve closer attention.

For a data analyst role, this project is strongest because it demonstrates three skills recruiters care about most: cleaning and structuring raw data, turning calculations into interpretable metrics, and communicating a technical topic in plain language for a broad audience.

Business / Research Problem

As the number of active satellites grows, orbital congestion increases the likelihood of close approaches and collision risk. The question behind this project is simple: how can raw orbital data be converted into a clear, usable view of where risk is concentrated and how operators should respond?

Data Collection and Cleaning

The project uses a live active-satellite TLE feed saved in the repository as active.txt. The file is organized as repeating three-line blocks: satellite name, line 1, and line 2 of the TLE record. The parser groups these lines into records, strips whitespace, and loads them into the Skyfield orbital model.

- Converted raw text lines into structured satellite records so they could be analyzed in a tabular format.
- Standardized orbital positions into latitude, longitude, altitude, and three-dimensional x/y/z coordinates.
- Prepared the output as a pandas DataFrame so it could be filtered, sliced, and displayed inside the dashboard.
- Used caching in Streamlit to avoid reprocessing the full dataset on every page interaction.

Analysis Approach

The analysis focuses on proximity detection. Each satellite is propagated to the current timestamp, then compared against other satellites using a standard Euclidean distance formula in three dimensions. Any pair that falls below the user-defined threshold is flagged as a conjunction risk and highlighted in the map.

- Position propagation: the Skyfield SGP4 model is used to estimate current orbital state from each TLE record.
- Distance calculation: satellites are compared pairwise using x, y, and z coordinates in kilometers.

- Threshold-based alerting: the sidebar control lets the user adjust how conservative the risk screen should be.
- Visual encoding: satellites involved in a flagged pair are colored red, while other objects remain cyan for quick scanning.

Key Findings

The repository already performs end-to-end data preparation.

The project does not merely display a map. It ingests raw orbital text, converts it into structured records, and transforms those records into analysis-ready coordinates.

Risk detection is explicit and explainable.

The conjunction logic uses a transparent distance rule, which makes the output easy to defend to non-technical reviewers and easy to tune for different risk tolerances.

The dashboard communicates the result clearly.

A live map plus alert list translates a technical calculation into a format that a recruiter, manager, or operations user can understand quickly.

The project has a clear decision-support angle.

Instead of being a static visualization, the dashboard helps answer a practical question: which objects are close enough to merit attention right now?

Recommendations / Actionable Insights

- Prioritize monitoring in the most crowded orbital bands by running the analysis at multiple thresholds and comparing how alert volume changes.
- Expand the report with trend views over time so the dashboard shows whether conjunction risk is increasing, stable, or decreasing.
- Add summary metrics such as total tracked objects, number of alerts, and most frequent satellite pairs to strengthen the executive-readout value.
- Include a short written narrative beside the visualization so the document clearly explains what the numbers mean and what action should follow.

Why this works for a data analyst application

This project fits the prompt because it shows a complete analytics workflow: raw data ingestion, cleaning and structuring, quantitative analysis, and a plain-English summary of what the results mean. It also stands out because the topic is unusual, which makes the sample memorable while still demonstrating core analyst skills.

Tools and Techniques Used

Tool / Method	How it was used
Python / pandas	Loaded, cleaned, and organized orbital records for analysis.
Skyfield / SGP4	Propagated TLE data into current orbital positions.
NumPy	Computed three-dimensional distances between satellite pairs.
Streamlit	Presented the analysis in an interactive application.
Folium / streamlit-folium	Rendered the map and highlighted conjunction risks visually.

Repository Components Referenced in This Sample

- app.py - Streamlit interface, data loading, alert display, and map rendering
- src/parse_tle.py - TLE parsing and orbital position computation
- src/risk.py - pairwise distance calculations and risk detection
- src/map.py - Folium map creation and risk highlighting
- active.txt - satellite TLE input data used by the project

Prepared for application use. This sample can be paired with dashboard screenshots or a GitHub link to provide a complete portfolio submission.